

Tokenization

Word · Character · Subword · BPE · WordPiece · SentencePiece

Outline

Why tokenize?

Byte-Pair Encoding

Other subword algorithms

Tokenization in practice

Play along: watch tokenization live

Open one of these now and paste text as we go:

Tiktokenizer `tiktokenizer.vercel.app`

live in the browser; switch GPT-2 / GPT-4 / Llama and see every token + its ID

OpenAI Tokenizer `platform.openai.com/tokenizer`

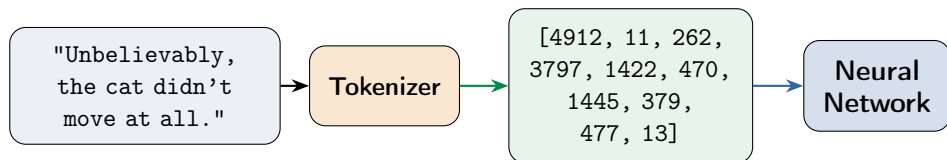
official; shows token and character counts for the GPT models

tiktoken (library) `github.com/openai/tiktoken`

OpenAI's fast byte-level BPE in Rust — `pip install tiktoken`

Paste a big number like 1234567, a non-English word, indented code, and a word with a leading space. Watch the token count jump around — we will explain every surprise by the end.

Models need numbers, not text



Tokenization is the first step in any NLP pipeline: split raw text into discrete units (tokens) and map each to an integer ID. The choice of tokenizer affects **everything** downstream.

Three levels of granularity

Word-level

["Unbelievably",
"the", "cat",
"didn't", "move"]

Vocab size: $\sim 100k+$

OOV problem

Subword

["Un", "believ",
"ably", ",", "the",
"cat", "didn", "'t"]

Vocab size: $\sim 30k-50k$

The sweet spot

Character

["U", "n", "b", "e",
"l", "i", "e", "v",
"a", "b", "l", "y", ...]

Vocab size: ~ 256 (bytes)

Very long sequences



Used by all modern LLMs

Word-level tokenization and the OOV problem

Vocabulary (fixed at training):

the, cat, sat, on, mat,
dog, run, happy, sad, ...

Size: 50,000–200,000 words

At test time:

"The cryptocurrency market
plummeted after the
CEO's tweet."



Words not in vocab → [UNK] [UNK]
market [UNK] after the [UNK] tweet.

- Problems:**
- New words, names, typos, or **casing** ("The" vs "the") → [UNK]
 - Huge vocabulary → huge embedding matrix
 - Morphology lost: "run", "runs", "running" are unrelated tokens

Character-level: no unknowns, but...

Input: "The cat sat on the mat."

The | cat | sat | on | the | mat | .

7 tokens (word-level)

T|h|e| |c|a|t| |s|a|t| |o|n| |t|h|e| |m|a|t|.|

23 tokens (character-level)

Drawbacks:

- Sequences $\sim 3\text{--}5\times$ longer (here $23/7 \approx 3.3\times$)
- $O(n^2)$ attention cost explodes
- Characters carry little meaning
- Harder long-range dependencies

Advantages:

- Tiny vocab (~ 256 byte values)
- Zero unknown tokens
- Works for any language
- Handles typos, code, URLs

Here "character" means a **byte** (~ 256 values); full Unicode has $>100k$ distinct symbols, which is why models go byte-level (next).
Too fine-grained alone — but it inspires **subword** methods.

Subword tokenization: the key insight

Common words stay whole: the, cat, and
Rare words are split into known pieces: un + believ + ably

“playing”  play + ing

“unhappiness”  un + happi + ness

“ChatGPT”  Chat + G + PT

“brrrr”  br + rr + r

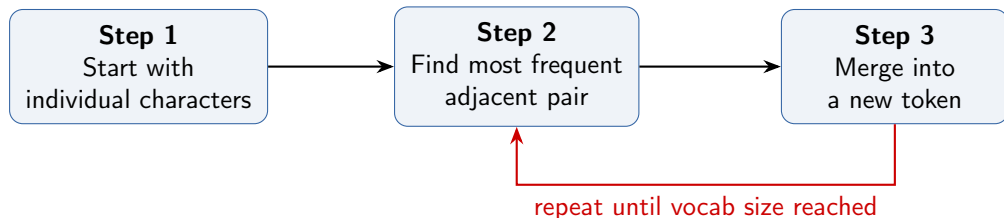
Vocab size ~30k–50k
able sequence lengths

- No [UNK] tokens
- Morphology is partially captured

• Reason-

Byte-Pair Encoding (BPE): the idea

Training: learn merge rules from a corpus. **Inference:** apply merge rules to new text.



Originally a **data compression** algorithm (Gage, 1994).
Adopted for NLP by Sennrich et al. (2016). Used
by **GPT**, **GPT-2**, **RoBERTa**, **BART**, **LLaMA**.

BPE: worked example

Training corpus (word $\times$ frequency): cheese $\times 10$, cheek $\times 8$, cheap $\times 5$

Initial vocab: c, h, e, s, k, a, p

Splits: cheese ($\times 10$) cheek ($\times 8$) cheap ($\times 5$)

Merge 1: most frequent pair = (c, h) freq =
 $10+8+5 = 23$

Splits: **ch**ease ($\times 10$) **ch**eck ($\times 8$) **ch**ea p ($\times 5$)

Merge 2: most frequent pair = (ch, e) freq =
 $10+8+5 = 23$

Splits: **che**ese ($\times 10$) **che**ek ($\times 8$) **che**a p ($\times 5$)

Merge 3: most frequent pair = (che, e) freq =
 $10+8 = 18$

Splits: **chee**se ($\times 10$) **chee**k ($\times 8$) cheap ($\times 5$)

Merge rules (ordered):

1. c + h $\rightarrow$ ch
2. ch + e $\rightarrow$ che
3. che + e $\rightarrow$ chee
- $\vdots$

Vocab after 3 merges:

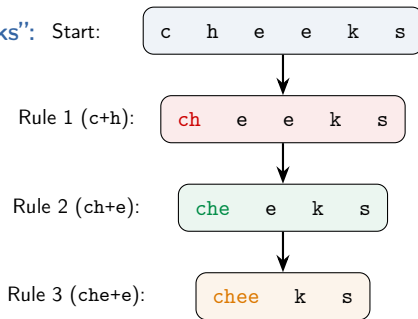
c, h, e, s, k, a, p,
ch, che, chee

“cheap” stops merging at step 3 (no “ee”). Note: at step 1, (c,h) and (h,e) are both freq 23 — BPE breaks such ties by a fixed order (here, the first pair seen).

BPE: tokenizing new text

Given the learned merge rules, tokenize a word **never seen** in training:

Tokenize “cheeks”: Start:



Key point:

Apply merge rules in the *same order* they were learned.

“cheeks” was never in the training corpus, but BPE splits it into known pieces: chee + k + s

Each ID = the token's index in the vocab.

Result: ["chee", "k", "s"] → token IDs [9, 4, 3]

BPE in code: counting adjacent pairs

Same algorithm as the worked example, now the canonical implementation (Sennrich et al., 2016). Each word is stored as space-separated symbols ending in `</w>`, an explicit **end-of-word** marker so a merge never crosses a word boundary.

```
import re, collections

def get_stats(vocab):
    pairs = collections.defaultdict(int)
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols)-1):
            pairs[symbols[i], symbols[i+1]] += freq
    return pairs
```

```
vocab = {'l o w </w>' : 5,
         'l o w e r </w>' : 2,
         'n e w e s t </w>' : 6,
         'w i d e s t </w>' : 3}

pairs = get_stats(vocab)
# ('e', 's'): 9, ('s', 't'): 9,
# ('t', '</w>'): 9, ('w', 'e'): 8,
# ('l', 'o'): 7, ('o', 'w'): 7 ...
```

get_stats counts each adjacent pair, weighted by word frequency. Top pair: (e, s) at 9 = 6(newest) + 3(widest).
Three pairs tie at 9; max(pairs, key=pairs.get) keeps the first one seen, (e, s).

BPE in code: applying one merge

`merge_vocab` glues the winning pair into a single symbol wherever it occurs. The regex matches the pair only as two *whole* symbols, so a merge never fuses across a space.

```
def merge_vocab(pair, v_in):
    v_out = {}
    bigram = re.escape(' '.join(pair))
    p = re.compile(r'(?!\S)' + bigram + r'(?!\S)')
    for word in v_in:
        w_out = p.sub(' '.join(pair), word)
        v_out[w_out] = v_in[word]
    return v_out

best = max(pairs, key=pairs.get)    # ('e', 's')
vocab = merge_vocab(best, vocab)    # 'e s' -> 'es'
```

Before:

```
new e s t </w> : 6
wid e s t </w> : 3
```



After merge (e, s):

```
new es t </w> : 6
wid es t </w> : 3
```

BPE in code: the full training loop

Wrap it in a loop: recount, pick the best pair, merge, repeat. One hyperparameter — the number of merges, which sets the final vocab size.

```
num_merges = 10

for i in range(num_merges):
    pairs = get_stats(vocab)
    best = max(pairs, key=pairs.get)
    vocab = merge_vocab(best, vocab)
    print(best)
```

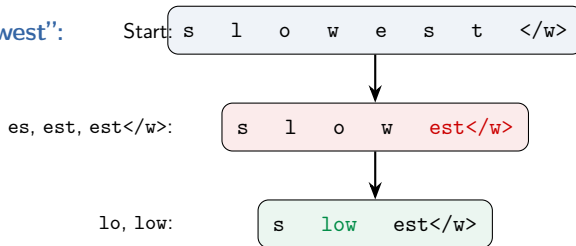
```
('e', 's')           # es
('es', 't')          # est
('est', '</w>')        # est</w>
('l', 'o')           # lo
('lo', 'w')          # low
('n', 'e')           # ne
('ne', 'w')          # new
('new', 'est</w>')    # newest</w>
('low', '</w>')        # low</w>
('w', 'i')           # wi
```

Merges build up in frequency order: first the suffix `est</w>`, then whole words `low`, `new`, `newest</w>`. BPE learns morphology and frequent words **automatically** — no linguistic rules.

Why the learned merges generalize

The 10 merges define the vocabulary's pieces. A word **never seen in training** still splits into known pieces — just apply the merge rules in the order they were learned.

Tokenize “slowest”:



Result: ["s", "low", "est</w>"] — 3 known pieces, 0 unknowns

`low` and `est</w>` were learned from *lower*, *newest*, *widest* — never from *slowest*. This is why a ~50k-merge BPE vocabulary covers essentially any input with no [UNK].

Everything is bytes: the UTF-8 encoding

Byte-level BPE starts from “256 byte values” — but where do bytes come from? **UTF-8** turns every character into **1 to 4 bytes**, depending on its Unicode code point.

Code point range	Bytes	Example (hex)
U+0000..U+007F (ASCII)	1	A → 41
U+0080..U+07FF	2	é → C3 A9 (also Armenian, Arabic, Hebrew)
U+0800..U+FFFF	3	CJK U+4E2D → E4 B8 AD
U+10000..U+10FFFF	4	emoji U+1F642 → F0 9F 99 82

ASCII stays 1 byte, so UTF-8 is **backwards-compatible** with plain English. Byte-level BPE starts from these 256 bytes, so it encodes **any** script with no [UNK] — but non-Latin text costs more from the start.

Armenian “barev” (4 letters) = 8 bytes = 8 GPT-4 tokens; English “hello” = **1 token**. Same greeting, **8×** the cost.

Byte-level BPE (GPT-2, GPT-3, LLaMA)

Standard BPE

Base vocab = Unicode characters

Vocab size = $\sim 30\text{k} - 50\text{k}$

Unknown chars $\rightarrow$ [UNK]

Problem: Chinese, emoji, etc.
can hit unknown characters

upgrade


Byte-level BPE

Base vocab = **256 byte values**

Vocab size = 256 + merges

(GPT-2: 256 bytes + 50,000
merges + 1 special = 50,257)

No [UNK] ever

Any byte sequence is representable

Every text is ultimately a sequence of bytes (UTF-8 encoding).
By starting from **bytes** instead of characters, BPE can tokenize
any input: English, Chinese, Arabic, code, emoji,
binary data — all with the same vocabulary.

Decoding: token IDs back to text

The inverse of encoding: look up each ID's bytes in the vocabulary, concatenate all the bytes, then UTF-8-decode into a string.



Gotcha: not every byte sequence is valid UTF-8. If a model emits a broken sequence, `bytes.decode("utf-8")` would *crash* — so real code passes `errors="replace"`, substituting a replacement character instead of failing.

Pre-tokenization: don't merge across boundaries

Left to itself, BPE would merge a frequent word with the punctuation beside it: dog., dog!, dog? each become their own token, tangling meaning with punctuation.

Naive BPE

input: dog. dog! dog?

→ dog. dog! dog?

Three near-duplicate tokens; dog is entangled with punctuation.

pre-split

Split first, then BPE

chunks: dog . _dog ! _dog ?

→ one dog token,
punctuation kept separate

Merges happen only *within* a chunk, never across.

GPT-2 first chops text into chunks with a regex, runs BPE **independently inside each chunk**, then concatenates. A merge can therefore never cross a letter ↔ number ↔ punctuation ↔ space boundary.

The GPT-2 split pattern (and what GPT-4 changed)

GPT-2 splits with this regex before any merging (the `regex` module, not `re`):

```
's|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+
```

Each `|` is an alternative, tried left to right:

- `'s 't 're 've ...` — contractions
- `\p{L}+` — letters (opt. leading space)
- `\p{N}+` — digits (opt. leading space)
- everything else — punctuation
- `\s+` — runs of whitespace

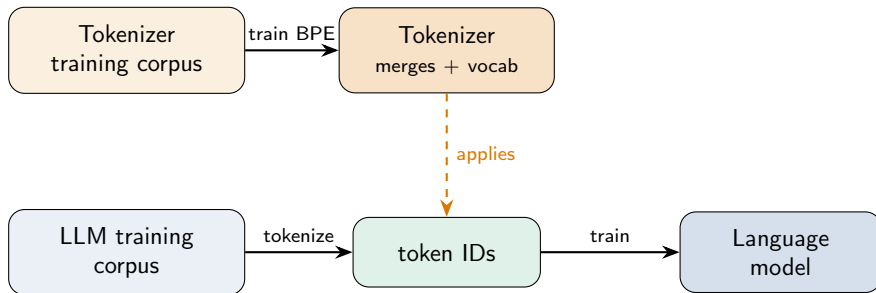
Example split:

```
"Hello world123 :)"  
→ ["Hello", "_world",  
   "123", "_:)" ]
```

GPT-4 changed two things: contractions match **case-insensitively**, and numbers are capped at **three digits** (`\p{N}{1,3}`) — so no single token is ever a long number. That alone measurably helps arithmetic.

The tokenizer is its own model

The tokenizer is trained **once**, as a preprocessing step, on its **own** corpus — a completely separate object from the language model.



The two corpora can **differ**. A string can be common in the *tokenizer's* data (so it earns its own token) yet rare or absent in the *model's* data (so that token is barely trained). **Remember this** — it is the root of the SolidGoldMagikarp bug we meet later.

WordPiece (BERT, DistilBERT)

BPE

Merge criterion:
most frequent pair

Greedy count-based

Used by: GPT, LLaMA, etc.

WordPiece

Merge criterion:
pair that **maximizes likelihood**
of the training corpus

Used by: BERT, DistilBERT

$$\text{score}(a, b) = \frac{\text{freq}(ab)}{\text{freq}(a) \times \text{freq}(b)}$$

$\approx$ PMI: rewards *surprising* co-occurrence

Notation: WordPiece marks *continuation* subwords with ##

"unbelievably" $\rightarrow$ ["un", "##bel", "##ie", "##va", "##bly"] (the real BERT split)

BPE instead marks *word-initial* subwords (e.g., GPT-2 uses Ġ = space prefix)

WordPiece: BPE vs WordPiece merge decision

Corpus (all split to characters): cheese ($\times 10$), cheek ($\times 8$), cheap ($\times 5$), seep ($\times 7$).
Which pair should we merge: (e, e) or (e, p)?

BPE: raw pair count

(e, e): cheese 10 + cheek 8 + seep 7
= 25

(e, p): seep 7 = 7

("cheap" is c-h-e-a-p: no adjacent e, p)

Winner: (e, e) — more frequent

WordPiece: likelihood score

score = $\frac{\text{freq}(ab)}{\text{freq}(a)\text{freq}(b)}$, freq(e) = 65,
freq(p) = 12

(e, e): $\frac{25}{65 \times 65} = 0.0059$

(e, p): $\frac{7}{65 \times 12} = 0.0090$

Winner: (e, p) — higher score

WordPiece favors pairs whose co-occurrence is
surprising given how common each piece is.

"e" is everywhere, so "ee" is expected; "ep" co-occurring is more informative.

Unigram LM and SentencePiece

Unigram LM

Start with a *large* vocab

Iteratively **remove** tokens
that hurt likelihood the least

Top-down (vs BPE's bottom-up)

Used by: T5, ALBERT, XLNet

Kudo, 2018

SentencePiece

Not an algorithm — a **library**

Treats input as raw byte stream
(no pre-tokenization needed)

Supports both **BPE** and **Uni-gram**

Language-agnostic: no need for
space-based word splitting

Kudo & Richardson, 2018

BPE = bottom-up (merge frequent pairs) vs

Unigram = top-down (prune unlikely tokens)

Unigram gives each token a probability and segments a word into the piece-sequence with the highest product of probabilities; training drops the pieces whose removal costs the least likelihood.

Both converge to similar subword vocabularies in practice.

Why tokenization matters: LLM quirks

Bad at arithmetic

"12345" $\rightarrow$ ["123", "45"]

The model never sees the individual digits together!

Poor non-English efficiency

English "hello" = 1 token

Armenian "barev" = 8 tokens

Same greeting, 8 $\times$ the cost!

Sensitive to formatting

"Hello World" and

"Hello World" produce different token sequences.

Can't count letters

"How many r's in strawberry?"

"straw" + "berry" — the model can't see individual letters.

Many apparent "reasoning failures" of LLMs are actually **tokenization artifacts**.
The model literally cannot see what you think it sees.

The foot-guns, up close

SolidGoldMagikarp — a Reddit username frequent enough in the *tokenizer's* data to earn its own token, but absent from the *model's* training data. Its embedding row was never updated — untrained, like unallocated memory. Ask GPT-2 about it and you get evasion, hallucination, even insults.

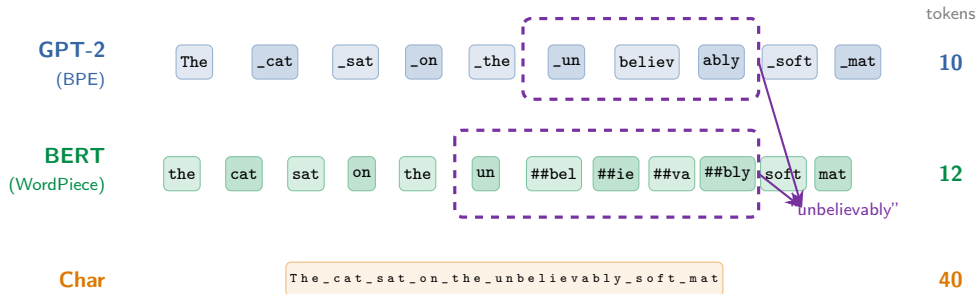
Trailing whitespace — `_world` (space + word) is one token. End a prompt with a bare space and you feed token 220 instead, splitting off that natural token. Now off-distribution — OpenAI literally warns a trailing space “causes worse performance”.

`<|endoftext|>` injection — a special-token *string* in user input can be parsed as the real special token, making the model abruptly halt or refuse. A small but real attack surface hiding in the tokenizer.

A lot of “LLM weirdness” traces straight back to tokenization. Don't brush it off.

Visualizing: the same sentence, different tokenizers

Input: "The cat sat on the unbelievably soft mat"



Same input, very different representations. "unbelievably"
= **3 tokens** (GPT-2), **5 tokens** (BERT), **12 characters**.
GPT-2's _ marks a leading space; BERT's ## marks a word continuation.
Fewer tokens = cheaper inference (less compute per sequence).

Special tokens

[CLS]

Classification
token (BERT)

[SEP]

Separator
between
segments

[PAD]

Padding to
equal length

[UNK]

Unknown token
(fallback)

⟨BOS⟩

Beginning
of sequence

⟨EOS⟩

End of
sequence

[MASK]

Masked
position
(BERT MLM)

Special tokens are **not** in the original text — they're added by the tokenizer to give the model structural signals: where sequences begin/end, what to predict, etc.

Choosing the vocabulary size: a tradeoff

Vocab size appears in exactly two places in the model: the **embedding table** (one row per token) and the **final softmax** (one output per token). So it is a real hyperparameter with a cost on both ends.

Too small (e.g. raw bytes, 256)

- sequences get very long
- less text fits in the context
- more compute per sentence

Too large (e.g. millions)

- huge embedding + softmax
- rare tokens are **undertrained**
- too much crammed per token

Sweet spot: modern LLMs use roughly **30k–100k** tokens. Bigger vocab = shorter sequences (more context) but fewer training examples per token. It is tuned empirically, not derived.

Comparison of tokenization methods

Method	Direction	Criterion	Vocab size	Used by
BPE	Bottom-up	Frequency	30k–50k	GPT, LLaMA
WordPiece	Bottom-up	Likelihood	30k	BERT
Unigram	Top-down	Likelihood	30k–50k	T5, XLNet
Byte BPE	Bottom-up	Frequency	50k–100k	GPT-2/3/4
Character	—	—	256	ByT5

In practice, the differences between BPE, WordPiece, and Unigram are **small**.

What matters most: vocab size, training corpus, and whether byte-level is used.

Byte-level BPE is the current default for new large language models.

Practical: tokenizers in action

```
# GPT-4 tokenizer (tiktoken:  
# OpenAI's fast BPE impl in Rust)  
import tiktoken  
enc = tiktoken.encoding_for_model(  
    "gpt-4")  
tokens = enc.encode(  
    "Hello world!")  
# [9906, 1917, 0]  
enc.decode(tokens)  
# "Hello world!"
```

```
# BERT tokenizer  
from transformers import  
    AutoTokenizer  
tok = AutoTokenizer.from_pretrained(  
    "bert-base-uncased")  
tok.tokenize(  
    "unbelievably")  
# ["un", "##bel", "##ie",  
#  "##va", "##bly"]
```

Try it yourself: <https://platform.openai.com/tokenizer>
Paste any text and see how GPT tokenizes it. Pay attention to:
numbers, non-English text, code, and whitespace.

Trend: byte-level tokenizers

Why go byte-level?

- **No unknown tokens** — ever
- Truly **language-agnostic**: same vocab for all scripts
- Handles code, URLs, binary
- No brittle *pre-tokenization* (the regex word-split done first)
- Simpler, more robust pipeline

Models & approaches

- **ByT5** (2022): pure byte-level, no merges at all
- **MegaByte** (2023): byte-level with patching for efficiency
- **GPT-4, LLaMA 3**: byte-level BPE with large merge tables
- Trend toward *fewer assumptions* about input structure

The trade-off: pure byte-level models produce **longer sequences** ($\sim 4\times$), increasing compute cost. Byte-level *BPE* (with merges) is the current sweet spot: byte-safe base vocabulary + subword compression.

Further reading

Subword Tokenization

- Sennrich et al. (2016), “Neural Machine Translation of Rare Words with Subword Units” (BPE)
- Kudo & Richardson (2018), “SentencePiece: A simple and language independent subword tokenizer”
- Kudo (2018), “Subword Regularization: Improving NMT Models with Multiple Subword Candidates” (Unigram LM)

WordPiece & Byte-Level

- Schuster & Nakajima (2012), “Japanese and Korean Voice Search” (WordPiece)
- Wang et al. (2020), “Neural Machine Translation with Byte-Level Subwords”

Practical Resources

- HuggingFace Tokenizers library documentation — huggingface.co/docs/tokenizers
- OpenAI Tiktoken (fast BPE in Rust) — github.com/openai/tiktoken
- OpenAI Tokenizer tool — platform.openai.com/tokenizer

Questions?

Next: Evaluation — Perplexity, BLEU, ROUGE